

۱. مقدمه و پیشینه

در دنیای امروز، مدل‌های زبانی بزرگ (LLM) به یکی از پایه‌های اصلی نرم‌افزارهای هوشمند تبدیل شده‌اند. این مدل‌ها قادرند متون طبیعی را درک کرده، آن‌ها را تحلیل کنند و پاسخ‌هایی دقیق و روان تولید نمایند. با این حال، یکی از بزرگ‌ترین چالش‌های این مدل‌ها آن است که دانش آن‌ها به زمان آموزش محدود می‌شود و دسترسی مستقیمی به داده‌های جدید، اختصاصی یا سازمانی ندارند.

برای رفع این محدودیت، رویکرد «بازیابی-افزوده تولید» یا RAG (Retrieval-Augmented Generation) مطرح شد. این رویکرد ترکیبی از دو مرحله اصلی است:

1. در مرحله اول، اطلاعات مرتبط از یک پایگاه دانش بیرونی بازیابی می‌شود.
2. در مرحله دوم، مدل زبانی از این اطلاعات بازیابی شده به عنوان زمینه (Context) استفاده می‌کند تا پاسخ نهایی را تولید نماید.

معماری RAG نخستین بار توسط لوئیس و همکارانش در سال ۲۰۲۰ معرفی شد و از آن زمان تاکنون تحولات گسترده‌ای را تجربه کرده است. امروزه این معماری در طیف وسیعی از کاربردها، از جمله پرسش و پاسخ سازمانی، چت‌بات‌های پشتیبانی مشتری، تحلیل اسناد حقوقی، سیستم‌های پزشکی و هوش تجاری (BI) مورد استفاده قرار می‌گیرد.

۲. اجزای اصلی یک سیستم RAG

یک سیستم RAG از چندین مؤلفه کلیدی تشکیل شده است که هر یک نقش مهمی در کیفیت نهایی پاسخ‌ها ایفا می‌کنند.

۲.۱ پردازش و بارگذاری اسناد (Document Ingestion)

اولین گام در ساخت یک سیستم RAG، آماده‌سازی و بارگذاری اسناد است. در این مرحله، متن خام از اسناد استخراج شده، پاک‌سازی می‌شود و برای مراحل بعدی آماده می‌گردد. اسناد می‌توانند در قالب‌های مختلفی مانند PDF، Word، HTML، متن ساده و حتی پایگاه‌های داده ساخت‌یافته ارائه شوند. پس از استخراج محتوا، داده‌ها پردازش شده و برای استفاده در مراحل بعدی سیستم آماده می‌شوند.

۲.۲ تقسیم‌بندی متن (Chunking)

پس از بارگذاری اسناد، متن‌ها به قطعات کوچک‌تری به نام Chunk تقسیم می‌شوند. این کار به دلیل محدودیت پنجره زمینه (Context Window) مدل‌های زبانی ضروری است.

استراتژی‌های متداول تقسیم‌بندی عبارت‌اند از:

الف) تقسیم‌بندی با اندازه ثابت (Fixed-size Chunking)

در این روش، متن به قطعاتی با تعداد ثابتی از کاراکترها یا توکن‌ها تقسیم می‌شود. این روش ساده و سریع است، اما ممکن است جملات یا پاراگراف‌ها را از یکدیگر جدا کرده و انسجام معنایی متن را کاهش دهد.

ب) تقسیم‌بندی معنایی (Semantic Chunking)

در این رویکرد، قطعات بر اساس مرزهای معنایی مانند پاراگراف‌ها، بخش‌ها یا موضوعات ایجاد می‌شوند. این روش کیفیت بالاتری دارد، زیرا ارتباط مفهومی میان اجزای متن حفظ می‌شود، اما پیاده‌سازی آن پیچیده‌تر است.

ج) تقسیم‌بندی با هم‌پوشانی (Overlapping Chunks)

برای حفظ زمینه در مرزهای بین قطعات، از هم‌پوشانی استفاده می‌شود. به‌عنوان مثال، اگر اندازه هر قطعه ۵۰۰ توکن باشد، ممکن است ۵۰ توکن انتهایی هر قطعه در ابتدای قطعه بعدی نیز تکرار شود. این روش به حفظ پیوستگی اطلاعات کمک می‌کند.

۲.۳ تبدیل به بردار (Embedding)

مرحله بعدی، تبدیل هر قطعه متن به یک بردار عددی چندبعدی است. این بردارها نمایانگر معنای متن در فضای برداری هستند و امکان مقایسه معنایی میان متون مختلف را فراهم می‌کنند.

از جمله مدل‌های مشهور Embedding می‌توان به موارد زیر اشاره کرد:

• OpenAI text-embedding-ada-002

• Sentence-BERT

• multilingual-e5

• paraphrase-multilingual-mpnet-base-v2

برای زبان فارسی، انتخاب مدل Embedding اهمیت بسیار زیادی دارد؛ زیرا بسیاری از مدل‌های عمومی عملکرد ضعیف‌تری روی متون فارسی دارند. مدل‌هایی مانند ParsBERT که برای زبان فارسی نتایج بهتری ارائه می‌دهد و برخی مدل‌های بومی ایرانی برای این زبان پیشنهاد می‌شوند.

۲.۴ پایگاه داده برداری (Vector Store)

الگوریتم‌های جستجو در پایگاه‌های داده برداری معمولاً بر اساس معیارهایی مانند شباهت کسینوسی (Cosine Similarity) یا فاصله اقلیدسی (Euclidean Distance) عمل می‌کنند. برای مقیاس‌های بزرگ، از الگوریتم‌هایی نظیر HNSW (Hierarchical Navigable Small World) و IVF (Inverted File Index) استفاده می‌شود که جستجوی تقریبی نزدیک‌ترین همسایه‌ها (ANN - Approximate Nearest Neighbor) را با سرعت بسیار بالا انجام می‌دهند.

۲.۵ بازیابی و رتبه‌بندی (Retrieval & Reranking)

هنگامی که کاربر سؤالی مطرح می‌کند، ابتدا سؤال به یک بردار عددی تبدیل می‌شود. سپس این بردار در پایگاه داده برداری جستجو شده و نزدیک‌ترین قطعات متن به آن بازیابی می‌شوند. در این مرحله معمولاً k قطعه برتر (Top-K) انتخاب می‌شوند. برای افزایش دقت بازیابی، می‌توان از مرحله‌ای به نام رتبه‌بندی مجدد (Reranking) استفاده کرد. در این مرحله، قطعات بازیابی‌شده توسط مدل‌های تخصصی‌تر ارزیابی و مجدداً مرتب می‌شوند. برخی از مدل‌های رایج برای رتبه‌بندی مجدد عبارت‌اند از:

- Cross-Encoder

- مدل‌های Rerank شرکت Cohere

- مدل‌های Rerank شرکت Anthropic

استفاده از این مرحله می‌تواند کیفیت نهایی پاسخ را به شکل قابل توجهی بهبود دهد.

۲.۶ تولید پاسخ (Generation)

در مرحله نهایی، سؤال اصلی کاربر به همراه قطعات بازیابی شده به عنوان Prompt به مدل زبانی بزرگ ارسال می شود.

ساختار Prompt معمولاً شامل سه بخش است:

1. دستورالعمل سیستم (System Prompt)

2. زمینه یا اطلاعات بازیابی شده (Retrieved Context)

3. سؤال کاربر

مدل زبانی با استفاده از این اطلاعات، پاسخی جامع، دقیق و مستند تولید می کند.

۳. چالش های RAG برای زبان فارسی

پیاده سازی RAG برای زبان فارسی با چالش های خاصی همراه است که در ادامه به مهم ترین آن ها اشاره می شود.

۳.۱ چندریختی متن فارسی

زبان فارسی دارای اشکال مختلف نوشتاری برای برخی کاراکترها است. برای مثال، حرف «ک» ممکن است به صورت «ك» (عربی) نیز نوشته شود. همچنین کاراکترهای «ی»، «ي» و «ئ» در برخی موارد معادل یکدیگر در نظر گرفته می شوند.

این موضوع می تواند باعث عدم تطابق متون هنگام جستجو و بازیابی اطلاعات شود. برای رفع این مشکل، لازم است پیش از تبدیل متن به بردار، مرحله ای تحت عنوان نرمال سازی متن (Text Normalization) انجام شود تا تمامی کاراکترها به یک قالب استاندارد تبدیل شوند.

۳.۲ اتصال کلمات و توکن‌بندی

فارسی یک زبان پیوندی (Agglutinative) است؛ به این معنا که پیشوندها و پسوندها به کلمات متصل می‌شوند. برای مثال، واژه «کتاب‌هایم» از اجزای زیر تشکیل شده است:

• کتاب

• ها

• ی

• م

این ویژگی باعث می‌شود فرایند توکن‌بندی (Tokenization) در زبان فارسی پیچیده‌تر باشد و به ابزارهای تخصصی نیاز داشته باشد. از جمله ابزارهای رایج برای این کار می‌توان به موارد زیر اشاره کرد:

• Hazm

• NLTK Persian

استفاده از این ابزارها به تفکیک صحیح اجزای کلمات و بهبود کیفیت بازیابی اطلاعات در سیستم‌های RAG کمک می‌کند.

۳.۳ کمبود داده‌های آموزشی

در مقایسه با زبان انگلیسی، حجم داده‌های آموزشی در دسترس برای زبان فارسی بسیار کمتر است. این موضوع کیفیت مدل‌های Embedding و مدل‌های تولید متن را تحت تأثیر قرار می‌دهد.

پروژه‌هایی مانند ParsBERT، FaBERT و mDeBERTa تلاش‌هایی برای کاهش این شکاف و بهبود عملکرد مدل‌های پردازش زبان فارسی به شمار می‌روند.

۳.۴ راست به چپ بودن متن

متن فارسی در رابط‌های کاربری و گزارش‌ها نیازمند پشتیبانی از نمایش راست به چپ (RTL – Right-to-Left) است. در پیاده‌سازی‌های فنی، باید اطمینان حاصل شود که تمامی مراحل پردازش، ذخیره‌سازی و نمایش داده‌ها از این ویژگی پشتیبانی می‌کنند.

عدم توجه به این موضوع می‌تواند منجر به نمایش نادرست متن، به هم ریختگی رابط کاربری و کاهش تجربه کاربری شود.

۴. معیارهای ارزیابی RAG

ارزیابی کیفیت یک سیستم RAG از چندین جنبه مختلف انجام می‌شود.

۴.۱ دقت بازیابی (Retrieval Accuracy)

این معیار نشان می‌دهد که آیا سیستم توانسته است قطعات مرتبط را با موفقیت بازیابی کند یا خیر.

شاخص‌های متداول در این حوزه عبارت‌اند از:

- Precision@K: دقت در K نتیجه اول
- Recall@K: فراخوانی در K نتیجه اول
- MRR (Mean Reciprocal Rank): میانگین رتبه معکوس
- NDCG (Normalized Discounted Cumulative Gain): معیار کیفیت رتبه‌بندی نتایج

این شاخص‌ها میزان موفقیت سیستم در یافتن اطلاعات مرتبط را اندازه‌گیری می‌کنند.

۴.۲ کیفیت تولید (Generation Quality)

این معیار کیفیت پاسخ نهایی تولیدشده توسط مدل زبانی را ارزیابی می‌کند.

از جمله شاخص‌های رایج برای سنجش کیفیت تولید می‌توان به موارد زیر اشاره کرد:

• BLEU

• ROUGE

• BERTScore

علاوه بر این، ارزیابی انسانی (Human Evaluation) نیز نقش مهمی در سنجش کیفیت پاسخ‌ها دارد.

چارچوب‌هایی مانند RAGAS نیز برای ارزیابی خودکار سیستم‌های RAG توسعه یافته‌اند و معیارهایی نظیر موارد زیر را اندازه‌گیری می‌کنند:

Faithfulness (صحت استناد)

Answer Relevancy (ارتباط پاسخ با سؤال)

Context Precision (دقت زمینه بازایی‌شده)

۴.۳ صحت استناد (Faithfulness)

یکی از مهم‌ترین معیارها در ارزیابی سیستم‌های RAG، اطمینان از این موضوع است که پاسخ تولیدشده واقعاً بر اساس محتوای بازایی‌شده باشد.

به بیان دیگر، مدل نباید دچار توهم‌زایی (Hallucination) شود؛ یعنی اطلاعاتی را ارائه دهد که در اسناد یا داده‌های بازیابی‌شده وجود ندارند.

این معیار به‌ویژه در حوزه‌های حساس مانند موارد زیر اهمیت بسیار زیادی دارد:

- پزشکی

- حقوقی

- مالی

در چنین کاربردهایی، کوچک‌ترین خطا یا ارائه اطلاعات نادرست می‌تواند پیامدهای جدی به همراه داشته باشد.

۵. معماری‌های پیشرفته RAG

۵.۱ HyDE و روش‌های پیشرفته RAG

در رویکرد HyDE (Hypothetical Document Embeddings)، ابتدا یک سند فرضی بر اساس پرسش کاربر تولید می‌شود و سپس از این سند برای جستجو در پایگاه داده استفاده می‌گردد. این روش باعث بهبود کیفیت بازیابی در مواردی می‌شود که خود پرسش به‌تنهایی اطلاعات کافی برای جستجوی دقیق ندارد.

علاوه بر HyDE، تکنیک‌های پیشرفته‌تری نیز در نسل جدید سیستم‌های RAG به کار می‌روند، از جمله:

- Query Rewriting (بهبود پرسش اولیه برای افزایش دقت بازیابی)

- Multi-hop Retrieval (بازیابی چندمرحله‌ای): انجام چند مرحله جستجو برای پاسخ به پرسش‌های پیچیده

ادغام نتایج چند مدل یا چند منبع بازیابی: Fusion of Retrieval Results

۵.۲ Agentic RAG

در این معماری، فرآیند بازیابی به صورت یک چرخه هوشمند و تکرارشونده انجام می شود. یک عامل (Agent) تصمیم می گیرد که:

- آیا بازیابی لازم است یا خیر

- از کدام منابع باید استفاده شود

- چند بار فرآیند بازیابی تکرار شود

ابزارهایی مانند LangGraph و CrewAI امکان پیاده سازی چنین سیستم های عاملی را فراهم می کنند.

۵.۳ Graph RAG

GraphRAG یک رویکرد نوین است که در آن به جای ذخیره صرف قطعات متن، یک گراف دانش (Knowledge Graph) از اسناد ساخته می شود.

در این گراف:

- موجودیت ها (Entities) از منابع مختلف استخراج می شوند

- روابط میان آن ها شناسایی و ذخیره می گردد

این ساختار برای پرسش هایی که نیاز به ترکیب چندین منبع اطلاعاتی دارند بسیار مؤثر است و دقت پاسخ دهی را افزایش می دهد.

۶. پیاده‌سازی عملی با LangChain

LangChain یکی از پرکاربردترین چارچوب‌ها برای ساخت سیستم‌های RAG است. در ادامه یک نمونه ساده از پیاده‌سازی این معماری ارائه می‌شود.

مرحله ۱: بارگذاری اسناد

در این مرحله از کلاس‌های loader مانند PyPDFLoader یا DirectoryLoader برای خواندن اسناد استفاده می‌شود.

مرحله ۲: تقسیم‌بندی متن

پس از بارگذاری، متن‌ها با استفاده از RecursiveCharacterTextSplitter به قطعات کوچک‌تر تقسیم می‌شوند. پارامترهای chunk_size و chunk_overlap باید متناسب با زبان فارسی تنظیم شوند.

مرحله ۳: تبدیل به بردار و ذخیره‌سازی

در این مرحله، هر chunk به یک بردار (Embedding) تبدیل شده و در پایگاه‌های داده برداری مانند Chroma یا FAISS ذخیره می‌شود.

مرحله ۴: ساخت زنجیره RAG

در نهایت، با استفاده از RetrievalQA یا LCEL (LangChain Expression Language)، زنجیره کامل بازیابی و تولید تعریف می‌شود.

۷. کاربرد RAG در هوش تجاری (BI)

RAG در حوزه هوش تجاری کاربردهای مهمی دارد، از جمله:

- تحلیل خودکار گزارش‌های سازمانی
- پاسخ‌گویی به پرسش‌های مدیریتی بر اساس داده‌های داخلی
- تولید داشبوردهای هوشمند با توضیحات متنی

- استخراج بینش (Insight) از داده‌های ساخت‌یافته و غیرساخت‌یافته
- این قابلیت‌ها باعث شده RAG به یکی از ابزارهای کلیدی در سیستم‌های BI مدرن تبدیل شود.

۸. بهترین رویکردها و توصیه‌ها

برای ساخت یک سیستم RAG فارسی با کیفیت بالا، بر اساس تجربه‌های عملی و مطالعات موجود، توصیه‌های زیر پیشنهاد می‌شود:

تمرکز جدی بر نرمال‌سازی متن فارسی

استفاده از کتابخانه‌هایی مانند Hazm برای یکسان‌سازی کاراکترها، حذف نیم‌فاصله‌های مشکل‌دار و اصلاح اشکالات نوشتاری ضروری است.

آزمایش مدل‌های مختلف Embedding

بهتر است مدل‌های کوچک‌تر (۲۵۶ تا ۵۱۲ توکن) و همچنین مدل‌های بزرگ‌تر (۵۱۲ تا ۱۰۲۴ توکن) متناسب با نوع داده تست شوند. انتخاب مدل باید بر اساس عملکرد واقعی در دامنه کاری انجام شود.

تنظیم دقیق chunk size

اندازه قطعات باید بر اساس نوع محتوا تعیین شود. برای متون فنی، chunk‌های کوچک‌تر معمولاً عملکرد بهتری دارند.

استفاده از Reranking برای پرسش‌های پیچیده

در پرسش‌های چندوجهی یا پیچیده، استفاده از مدل‌های Cross-Encoder یا سرویس‌های Rerank مانند Cohere یا Anthropic کیفیت نتایج را به‌طور قابل توجهی افزایش می‌دهد.

طراحی سیستم ارزیابی مستمر

برای سنجش مداوم (Golden QA Pairs) استفاده از مجموعه‌ای از سوالات و پاسخ‌های طلایی عملکرد سیستم ضروری است.

- استفاده از ابزارهای Observability
- ابزارهایی مانند LangSmith یا Langfuse برای پایش عملکرد، تحلیل خطاها و بهبود سیستم بسیار کاربردی هستند.

۹. آینده RAG

آینده RAG به سمت سیستم‌های پیشرفته‌تر و هوشمندتر در حال حرکت است، از جمله:

- Streaming RAG: تولید پاسخ به صورت تدریجی
 - Multimodal RAG: پردازش همزمان متن، تصویر، جدول و نمودار
 - RAG با حافظه بلندمدت: نگهداری تاریخچه مکالمات و استفاده از آن در پاسخ‌ها
- با افزایش ظرفیت مدل‌های زبانی و گسترش پنجره زمینه (Context Window)، برخی کاربردهای سنتی RAG ممکن است تغییر کنند، اما در سیستم‌هایی با داده‌های بزرگ و نیاز به به‌روزرسانی مداوم، همچنان نقش کلیدی خواهد داشت.

۱۰. نتیجه‌گیری

RAG یک معماری قدرتمند است که شکاف بین توانایی‌های مدل‌های زبانی بزرگ و نیازهای واقعی سازمان‌ها را پر می‌کند. با استفاده از این رویکرد می‌توان سیستم‌هایی ساخت که نه تنها بر دانش عمومی تکیه دارند، بلکه از داده‌های اختصاصی سازمان نیز به صورت دقیق و مستند استفاده می‌کنند.

با رعایت اصول مربوط به زبان فارسی، انتخاب مدل‌های مناسب، طراحی درست pipeline و ارزیابی مستمر، می‌توان سیستم‌های RAG با کیفیت بالا برای کاربران فارسی‌زبان توسعه داد.